

PITCH TO PROTOTYPE

A five-day unit where your class finds a real problem in your building, writes the spec for an app that fixes it, and watches you build it live on the board. The deliverable is working software, not a poster.

5

days, each with a clear end point

6

student pages worth printing

1

finished example app included

The grown-up holds the keyboard, and that is the point.

Students do the interviews, the problem statement, the feature list, the testing and the pitch. You do the prompting, projected, so they watch their own words become software. Specification and testing are the actual thinking — typing prompts is not.

Grades 4–8 · no coding background needed

You do not need to know how to code. You need to be willing to read what the AI produces, say "not quite", and ask again — in front of the class, which turns out to be the most valuable part.

What you need

A projector, a browser, and one AI chat account that *you* sign in to. Nothing is installed. Nothing is bought. Students never touch the chat — chatbots are 13 and up, and no child's name goes into a prompt at any point in this unit.

What comes out of it

A single web page that your school can actually use, saved as one file in your own Drive. It keeps its data in the browser: nothing is sent anywhere, there is no account, and there is no server to pay for or shut off in June.

Inside

Five teacher day-plans with scripts and the exact prompts · a rescue page for when the build breaks in front of the class · six printable student pages · every prompt collected on one page · and **Lost and Found**, a finished example app built by exactly this process, included as a file you can open today.

BEFORE YOU BEGIN

The five days at a glance

Each day ends somewhere definite. If a day does not reach its end point, do not move on — the next day depends on it.

DAY 1

Find a real problem

Students: interview three people in the building about something that annoys them. Not "an app idea" — a problem.

You: model one interview badly, then well. The difference is the lesson.

Ends with: every group holding three interview sheets.

DAY 2

Define it

Students: turn the interviews into one problem statement and sort features into must-have and nice-to-have.

You: push back hard on the must-have list. Three items, not eleven.

Ends with: one page per group that you could hand to a stranger.

DAY 3

Build it live

Students: read the spec aloud, watch it become software, call out what is wrong.

You: prompt on the board. Let them see it fail and get fixed.

Ends with: something that runs, even if it is ugly.

DAY 4

Test it on real people

Students: watch someone use it without explaining anything.

You: teach the one rule — if you have to explain it, that is a finding.

Ends with: a list of things real users got stuck on.

DAY 5

Pitch and change one thing

Students: pitch with evidence from testing; decide what to change and what to leave.

You: make one live change from their decision.

Ends with: a shipped tool the school can actually use.

Shorter on time?

Days 1 and 2 can be one long session. Day 4 cannot be skipped — it is where the learning is, and it is the day most units leave out.

WHY THIS UNIT WORKS

What students are actually practicing

If you have run a business unit before, this one will feel different. Here is what changed and why it matters.

The usual business unit	This one
Students invent a product nobody asked for.	Students find a problem somebody actually has, by asking.
The deliverable is a poster or a slide deck.	The deliverable runs, and the school keeps using it.
Success is measured by the pitch.	Success is measured by whether a stranger can use the thing.
Feedback is the teacher's opinion.	Feedback is watching a real user get stuck.

The single biggest shift is day 4.

Most units end at the pitch, which teaches students that a good presentation is the goal. Testing on a real user teaches them that the goal is the thing working — and that their own opinion of their design is not evidence.

What the students take away that lasts

How to ask about a problem without leading the answer. How to tell a must-have from a want. How to watch someone struggle without rescuing them. How to change your mind because of evidence. None of those are about apps.

Why you do the typing

The rule

The adult holds the keyboard. Always. Students never type into the AI chat, and nothing about a student — no name, no work, no photo, no description that could identify one — goes into a prompt.

The practical reason

The main AI chat tools require users to be 13 or older, and most of them are not designed for a class account. Putting a nine-year-old at that keyboard is a problem whatever your intentions.

The better reason

Prompting is the easy part and it is not the thinking. Deciding what to build, saying it precisely enough that someone else could build it, and judging whether what came back is right — that is the work, and it is all on the students. **Handing them the keyboard would mean handing them the shallow half.**

How it looks in the room

Your screen is projected. A student reads their spec aloud. You type what they said — not what you think they meant. When it comes back wrong, you ask them what is wrong, and you type their correction. **You are their hands, not their brain.**

Say the rule out loud on day 1 and give the reason. Students accept a boundary far better when it comes with an explanation, and "prompting is the easy part, I am not letting you skip the hard part" is an explanation they respect.

DAY 1

Find a real problem

The whole unit stands or falls here. A weak problem produces a pointless app and five days of going through the motions.

Start by banning the good idea

Tell them: **nobody pitches an idea today**. Today you only collect complaints. Ideas come from problems, and a problem you invented at your desk is not a problem — it is a guess.

Model it badly first — this is worth the five minutes

Interview a volunteer in front of the class, doing it wrong on purpose:

"Wouldn't it be good if there was an app for the lost property box?" — leading. They will agree with anything.

Then do it right: *"Tell me about the last time you lost something at school. What happened? What did you do? How did that go?"* Ask them which version told you more.

Who they interview

Three people, and **at least one adult who is not a teacher** — the office staff, a custodian, a lunch supervisor. Those three know more about what does not work in a building than anyone, and they are almost never asked.

Day 1 ends when

Every group has three completed interview sheets and has not yet suggested a single solution. If a group has already designed an app, send them back out — kindly, but send them.

Define it

Turning three messy conversations into one sentence a stranger could act on.

The problem statement frame

Students fill in: "[Who] needs a way to [do what], because [why it is hard now]." One sentence. If it needs an "and", it is two problems and they must pick one.

Then the sort, and this is where you push

Every group will produce eleven must-haves. Your job is to get them to three.

Ask of each one: **"If we shipped without this, would it still solve the problem?"** If yes, it is a nice-to-have. Say out loud that this is the hardest thing professionals do and most of them are bad at it too.

What a good one looks like

Problem: "The office staff need a way to know what is in the lost property box, because right now they have to tip the whole thing out every time a parent calls."

Must-have: add an item · see the list · mark it claimed. **Nice-to-have:** photos, categories, dates, email alerts, a search box.

Day 2 ends when

Each group has one page: a problem statement, three must-haves, and everything else parked in a nice-to-have column. That page is the spec. Tomorrow you build from it, word for word.

Do not let them add features tomorrow. The discipline of shipping what was specced is half the lesson, and it is the half adults find hardest.

DAY 3

Build it live

The day they see their own sentences turn into software. Protect it — do not let it become a lecture about AI.

How to run the room

Project your screen. A student from the group reads the spec aloud. **You type what they said, not what you think they meant.** If the spec is vague, the result will be wrong, and that is a better lesson than you quietly fixing it.

THE TEACHER PROMPT · USE THIS SHAPE EVERY TIME

Build a single HTML file that runs in a browser with no login and no internet. It is for [who], to help them [do what].

It must do exactly three things:

1. [must-have one]
2. [must-have two]
3. [must-have three]

Save everything to local storage on this device so it is still there tomorrow, and keep working with empty data if saving is unavailable. Big text and big buttons. Do not ask for anyone's name and do not collect any personal information.

Why that last sentence is in the prompt every single time

Left to itself the AI will helpfully add a "your name" field to almost anything. Saying no up front is easier than noticing it later — and doing it in front of the class, out loud, teaches them that this is a thing you decide on purpose.

Day 3 ends when

Something runs. It can be ugly. It can be missing a must-have. It has to run.

When it breaks in front of everyone

It will. The AI will produce something that does not work, mid-lesson, with thirty children watching. **This is the most valuable ten minutes in the unit** and it only works if you do not panic.

What not to do

Do not apologize for the technology. Do not switch to a backup slide deck. Do not say "let me fix this at lunch." All three teach the class that broken things are shameful and should be hidden.

What to do instead – three moves

1. **Name it out loud.** "That is not what we asked for. Look — we said mark it claimed, and it deleted the whole row."
2. **Ask them what they saw.** Not what to do about it — what they *saw*. Getting a class to describe a bug precisely is a genuine skill and they are better at it than you expect.
3. **Fix it in front of them.** Type their description back: *"When I click mark claimed it deletes the item. It should keep the item and show it as claimed."* Watch it come back right.

What they learn from those ten minutes: that adults do not actually know how everything works, that being stuck is ordinary rather than embarrassing, and that describing a problem clearly is most of solving it. **You cannot teach that from a slide.**

If three rounds have not fixed it

Start a fresh chat and paste the original prompt with the one thing that went wrong added. Say to the class why you are doing it: the conversation got tangled and starting clean is faster than untangling. That is also a real professional habit.

DAY 4

Test it on real people

The day most units skip, and the day the actual learning happens.

The one rule, and it is hard

If you have to explain it, that is a finding. Students hand the tool to a real user, say "can you add a lost umbrella", and then **say nothing**. Not one word. Watching someone struggle with something you made, without rescuing them, is genuinely difficult for a ten-year-old and that is the point.

Who tests it

The people they interviewed on day 1. Not each other — they already know how it works, which makes them useless as testers. Three real users is plenty; you will get every major finding from the first two.

What students record

Only what happened: where the user paused, what they clicked first, what they said out loud, what they got wrong. **Not opinions, and not what the student thinks the user should have done.** The observation sheet is deliberately narrow about this.

The debrief question that does the work

"You knew exactly how to use it. They did not. **What did you know that they did not?**" That question is the entire discipline of design, and eleven-year-olds get it immediately once they have watched someone fail in front of them.

Day 4 ends when

Each group has a list of things real users got stuck on, written as observations rather than excuses.

Pitch, then change one thing

The pitch is not the end. Shipping a change based on evidence is the end.

The pitch, four minutes

The problem, in the words of the person who has it · what we built · **what happened when a real person used it** · what we are changing because of that · what we are deliberately not changing, and why.

The part that separates this from every other pitch

"**What we are deliberately not changing.**" A group that says "a user wanted photos but that is a nice-to-have, so no" has understood something most adults in meetings have not. Reward it loudly.

Then make one change, live

Pick the highest-impact finding and fix it on the board in front of them. Five minutes. They watch their evidence become a change in the software, which is the whole arc of the unit compressed into one moment.

Then actually give it to the school

Save the file, put it on the office computer or the shared drive, and tell the class it is in use. **A tool that gets used is a different experience from a project that gets graded**, and they will remember which one this was.

If nothing is good enough to ship, say so honestly and explain what would need to be true. Pretending a weak tool is in use teaches them that the praise is automatic, which makes all your other praise worthless.

Problem hunt: interview sheet

Ask about a **problem**, never about an app. Fill one sheet per person.

Who I talked to (their job, not their name)

"Tell me about the last time something at school did not work the way it should."

"What did you do about it?"

"How often does that happen?"

The exact words they used that stuck with me

Check yourself before you finish

Did I suggest a solution? If yes, cross it out. Today is only for problems.

Problem statement and feature sort

Our problem, in one sentence

[Who] needs a way to [do what], because [why it is hard now].

MUST have – only three

Test: if we shipped without this, would it still solve the problem? If yes, it goes in the other column.

Nice to have – as many as you like

The hardest part of this page

Moving something you love into the right-hand column. Professionals find this hard too. Doing it on purpose is the skill.

Spec card – hand this to your teacher

This is what gets built. Whatever is not on this card does not get built, however much you want it. Print one per group and hand it over on day 3.

Group

Who is this for? (their job, not their name)

The problem, one sentence

It must do exactly these three things

Write them as things the user *does*, not as features. "Mark an item claimed", not "a claimed button".

Should it remember things after the page closes? Yes / No

If yes, say what:

Sign here, both of you

Group:

 Teacher:

Signing means: we build this, and we do not add to it mid-build.

Watch the build

While your teacher types, you are not a spectator. Write down what actually happened each round — this is the record of how your idea became real.

Round 1 — what we asked for

What we got

What was wrong with it

Round 2 — what we said next

What changed

Round 3 — what we said next

What changed

The question to answer at the end of the day

Which of our sentences was the vaguest, and how did we know?

If it broke

Write down exactly what you saw, in the fewest words that would let somebody else picture it.

Testing: what I saw

Say nothing

Hand it over, give them one task, then be quiet. If you have to explain it, write that down — **that is a finding, not a failure of the tester.**

The task I gave them

What they clicked first

Where they paused or looked confused

Anything they said out loud

Did they finish the task? Yes / No – and what got in the way

Afterwards, the big question

What did I know that they did not?

Pitch planner and decision log

1. The problem, in the words of the person who has it

2. What we built – the three must-haves

3. What happened when a real person used it

4. What we are changing, and why

5. What we are NOT changing, and why

Box 5 is the one that impresses people

Deciding not to build something, on purpose, with a reason, is harder and more valuable than adding one more feature.

THE EXAMPLE

What a finished one looks like

The pack includes a working app called `lost-and-found.html`. It is the example a real class would produce from a real spec — open it before you teach, so you know what the target looks like.

The spec it was built from

Problem: "The office staff need a way to know what is in the lost property box, because right now they have to tip the whole thing out every time a parent calls."

Must-haves: add an item · see the list · mark it claimed. That is all three, and the app does exactly those three things.

What it does

Add an item with where and when it was found · filter by waiting, claimed or everything · mark claimed and undo · flags anything sitting over 30 days · counts at the top · prints as a clean list for a clipboard · and it remembers everything between sessions.

What it deliberately does not do

There is nowhere to write a person's name. When an item is claimed you mark the item, not the claimer. That was the class rule, and it is the reason this tool can sit on the office computer without anyone having a conversation about it.

Use it as a worked example, not a template to hand out. If you give students the finished app on day 1 they will build backwards from it. Show it on day 5, after theirs exists, and ask what theirs does better.

Every teacher prompt, in one place

You type these, never a student. Square brackets mean put the group's own words in.

1 • THE BUILD, DAY 3

Build a single HTML file that runs in a browser with no login and no internet. It is for [who], to help them [do what]. It must do exactly three things: 1. [must-have] 2. [must-have] 3. [must-have]. Save everything to local storage on this device so it is still there tomorrow, and keep working with empty data if saving is unavailable. Big text and big buttons. Do not ask for anyone's name and do not collect any personal information.

2 • WHEN IT DOES THE WRONG THING

When I [what you clicked], it [what happened]. It should [what the spec says]. Fix that and give me the whole file again.

3 • THE DAY-5 CHANGE

We tested this with real users and they got stuck on [what happened]. Change [the one thing] so that [what should happen instead]. Do not change anything else.

4 • MAKE IT USABLE BY EVERYONE

Make sure this works for someone using a keyboard instead of a mouse: everything reachable with Tab and usable with Enter, with a clearly visible outline on the selected item. Do not use color alone to show status. Give every icon-only button a label a screen reader can announce.

5 • MAKE IT PRINTABLE

Add a print stylesheet so printing gives a clean list with no buttons and no colors — something that works on a clipboard.

6 • WHEN THREE ROUNDS HAVE NOT FIXED IT

[Start a fresh chat.] Paste prompt 1 again, then add: Also make sure that [the thing that kept going wrong].

Prompt 4 is not optional. If the tool goes into the office, someone with a motor or vision difference may have to use it. Asking costs one prompt.

Saving it, and what happens after the unit

Getting it out of the chat

Copy all the code with the copy button · paste into Notepad or TextEdit set to plain text, **not Word** · save as something like `lost-and-found.html`, setting "Save as type" to All Files on Windows · double-click to open. If it opens as a page of code, the `.html` did not save.

Where it should live

On the computer of whoever has the problem. The data saves in that browser on that machine — nothing is uploaded, no account exists. Moving the file to a different computer gives you a working but empty copy.

What would change if it went online

Hosting it would mean the data lives on someone's server and a link exists that could reach it. For a list of umbrellas that is not dramatic, but it becomes school software rather than a classroom tool, and that is a conversation with your technology team rather than a thing to do quietly. **Keeping it local is why this unit needs nobody's permission.**

The best outcome

Somebody in the building uses it in March without thinking about it, and a student walks past the office and sees their thing being used. That is worth more than any grade you could put on this.

How this unit is built

What we are claiming

That the prompts here, used with a current AI chat, produce working classroom tools from a class's own specification. The included example app was built this way and tested in a real browser before publication — items added, filters checked, claimed and undone, the page reloaded to confirm the data survived, the over-30-days flag verified against real dates, and the console checked for errors.

What we are not claiming

Not a computer science curriculum and not aligned to any CS standards. Not a replacement for your district's systems. Not legal advice about student data — your district's policy, your state's law, FERPA and COPPA are the authorities. And the AI tools change, which is why this unit teaches the loop and the discipline rather than one product's menus.

An honest limitation

Five days is enough to build something real and not enough to build something polished. Groups will finish with tools that work and look homemade. **Resist making them prettier** — the lesson is that a working ugly thing beats a beautiful idea, and that lesson is undermined by a teacher tidying it up overnight.

Where the student-facing design ideas come from

The problem-interview arc, the must-have sort and the watch-without-explaining testing rule are standard practice in design and product work, adapted here for grades 4–8. The tools named as worth building are drawn from 2026 reporting on what teachers already build for themselves with AI.

Built and audited by two certified teachers. Every prompt run, the example app built from the spec printed in this pack and tested in a browser before shipping.

If this was useful

Leaving feedback earns you credit

TPT gives you credit toward future purchases every time you review something you have downloaded. Go to **My Purchases**, find this unit, and leave a rating and a comment.

It also tells us what to build next, which is genuinely how these decisions get made.

If something did not work

Message us through TPT before leaving a rating and we will fix it and re-upload the same day, and you keep the updated file free. A prompt that stopped behaving is something we want to know about.

What your class will have done by Friday:

Asked a real person about a real problem. Written a specification precise enough for someone else to build from. Watched their own words become software. Handed it to a stranger and watched them struggle without rescuing them. Changed it because of evidence rather than opinion. **The app was never the point.**

Built and audited by two certified teachers

Bright Scholar · AI-Ready School